

LLM PRINCIPLES IN 30 DAYS

30天熟练 LLM 原理 教学与实践手册

一份给工程师的中文学习路线：从反向传播、Token、Embedding、Attention、Transformer 到手写 TinyGPT。每天 2 小时，目标是能讲清楚、能跑通、能修改一个小 GPT。

教学讲解

原理解释

可运行代码

30天任务

最终项目

适合：JS/Node 工程师、Python 初学者、想转向 AI Agent / LLM 工程的人。

建议本地目录： `src/me/learnAIagent/llm-30days`

目录

1. 这 30 天最终要达到什么程度	7. 第 3 周: Attention 与 Transformer
2. 学习方法和每天 2 小时节奏	8. 第 4 周: 手写 TinyGPT
3. 环境准备	9. 30 天每日计划
4. LLM 核心总地图	10. 最终项目验收标准
5. 第 1 周: 神经网络如何学习	11. 附录 A: 完整 TinyGPT 代码
6. 第 2 周: 语言如何变成数字	12. 附录 B: 资源与延伸阅读

1. 这 30 天最终要达到什么程度

30 天的目标不是训练一个 ChatGPT，也不是把所有论文都读完。目标是熟练掌握 LLM 的核心原理：你能把一个 GPT 从输入到输出的主链路讲清楚，能知道训练时每个关键模块在做什么，能跑通并修改一个很小的 GPT。

能讲清楚

不用资料解释：文本如何变成 token，token 如何变成向量，Transformer 如何混合上下文，模型如何输出下一个 token 概率。

能看懂代码

读懂 minGPT/nanoGPT 的核心文件，知道 CausalSelfAttention、Block、GPT.forward、generate 分别负责什么。

能跑通实验

准备一个 input.txt，用 PyTorch 训练 TinyGPT，看到 train loss / val loss 变化，并生成文本。

能修改参数

理解 block_size、n_embd、n_head、n_layer、learning_rate 对效果和速度的影响。

最终验收一句话

你应该能清楚解释：为什么输入一句话后，LLM 可以一步一步生成后面的 token？

2. 学习方法和每天 2 小时节奏

不要只看视频，也不要一开始就读论文。最有效的方式是：先用最小代码建立直觉，再把直觉映射到 Transformer 和 GPT。

每天时间	做什么	为什么
40 分钟	看讲解/读资料	建立当天概念，避免盲写代码。
70 分钟	敲代码/改参数/跑实验	LLM 原理必须通过张量形状和训练日志来理解。
10 分钟	写本地笔记	把当天学到的内容沉淀成自己的语言。

本地目录建议

把所有代码和笔记都放在 `src/me/learnAIagent/llm-30days` 。每天一个 md 文件，例如 `day01-next-token.md` 。

3. 环境准备

你可以继续用 macOS。本手册的实践代码使用 Python + PyTorch，因为深度学习训练生态主要围绕 Python。你 JS 很强，这会帮助你理解工程结构；Python 部分只需要掌握基本语法和 PyTorch 张量操作。

```
# 推荐在 src/me/learnAIagent/llm-30days 下做实验
mkdir -p ~/src/me/learnAIagent/llm-30days
cd ~/src/me/learnAIagent/llm-30days

# 建议使用 Python 3.11 或 3.12，新建独立环境
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy torch

# 验证
python - <<'PY'
import torch, numpy as np
print('torch', torch.__version__)
print('numpy', np.__version__)
print('device', 'cuda' if torch.cuda.is_available() else 'cpu')
PY
```

关于你之前遇到的 NumPy 警告

如果看到 `Failed to initialize NumPy: No module named numpy`，通常先在当前虚拟环境里执行 `python -m pip install numpy`。训练 LLM 时，PyTorch、NumPy、Python 版本最好放在独立 venv，避免和其他项目混在一起。

4. LLM 核心总地图

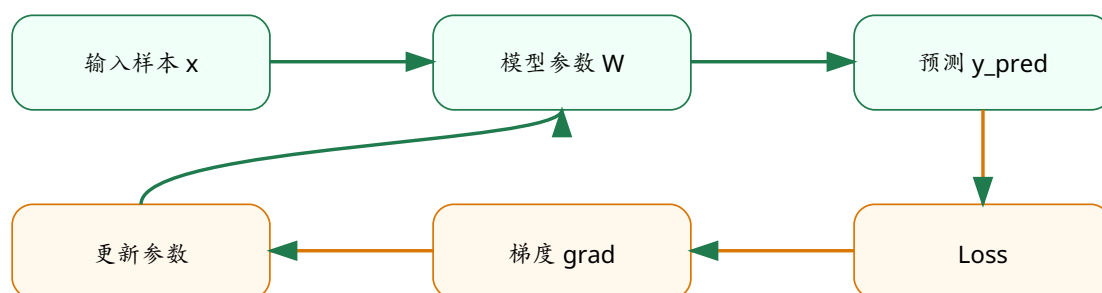
LLM 一次前向推理的主链路



图 1: LLM 的核心不是“查资料”，而是把上下文转换为下一个 token 的概率分布。

LLM 可以先理解为一个巨大的函数。它接收一串 token，输出下一个 token 的概率分布。训练时，我们已经知道正确的下一个 token，所以可以计算 loss；再通过反向传播调整参数。推理时，没有正确答案，只能把模型给出的概率分布采样成下一个 token，再拼回上下文继续预测。

训练闭环：模型为什么会变好



训练就是重复这个闭环：forward 得到预测，loss 衡量差距，backward 找到每个参数的责任，optimizer 修改参数。

图 2: 从“算错了”到“知道怎么改”，中间靠的是反向传播和优化器。

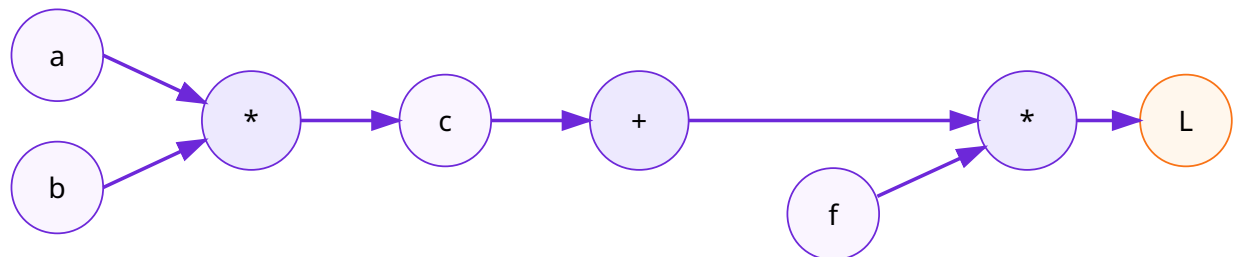
主线不要丢

无论你学到 Attention、LayerNorm、Residual、MLP、Optimizer，最终都要放回这条链：文本 -> token -> embedding -> transformer -> logits -> loss/backward 或 generate。

5. 第 1 周：神经网络如何学习

第一周不要急着碰 Transformer。先解决你之前一直追问的核心问题：参数到底为什么会自己变好？为什么不是只调最后一层？grad 正负代表什么？

计算图例子： $L = (a * b + c) * f$



forward 从左到右算数值；backward 从右到左算每个变量对最终 loss 的影响。

图 3：反向传播不是玄学，它只是链式法则在计算图上的系统化执行。

5.1 参数、变量、中间值、loss 的区别

概念	直觉	在 LLM 里的例子
变量	每次输入或计算过程中临时出现的值	某个 token 的 hidden state
参数	训练要长期保存和更新的数	Embedding 表、Attention 里的 $W_q/W_k/W_v$ 、MLP 权重
中间值	由变量和参数计算出来，帮助下一步计算	attention score、softmax 权重
loss	模型错得有多离谱的分数	正确下一个 token 概率太低时，cross entropy 变大

5.2 最小自动求导代码

下面这段代码模拟了一个极小版 autograd。你不需要一次背下来，但必须理解：每个操作都会保存 backward 规则，最后从输出节点反向调用。

```

class Value:
    def __init__(self, data, _children=(), _op=''):
        self.data = data
        self.grad = 0.0
        self._backward = lambda: None
        self._prev = set(_children)
        self._op = _op

    def __add__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data + other.data, (self, other), '+')
        def _backward():
            self.grad += 1.0 * out.grad
            other.grad += 1.0 * out.grad
        out._backward = _backward
        return out

    def __mul__(self, other):
        other = other if isinstance(other, Value) else Value(other)
        out = Value(self.data * other.data, (self, other), '*')
        def _backward():
            self.grad += other.data * out.grad
            other.grad += self.data * out.grad
        out._backward = _backward
        return out

    def backward(self):
        topo = []
        visited = set()
        def build(v):
            if v not in visited:
                visited.add(v)
                for child in v._prev:
                    build(child)
            topo.append(v)
        build(self)
        self.grad = 1.0
        for node in reversed(topo):
            node._backward()

# L = (a * b + c) * f
a, b, c, f = Value(2.0), Value(-3.0), Value(10.0), Value(-2.0)
L = (a * b + c) * f
L.backward()
print('L =', L.data)
print('a.grad =', a.grad, 'b.grad =', b.grad, 'c.grad =', c.grad, 'f.grad =', f.grad)

```

5.3 第 1 周检查点

- ☐ 能手算 $L=(a*b+c)*f$ 的 forward。
- ☐ 能解释 backward 为什么从最终 loss 往前走。
- ☐ 能解释 $\text{grad}>0$ 和 $\text{grad}<0$ 的含义。
- ☐ 知道参数更新公式 $p = p - \text{learning_rate} * \text{grad}$ 。
- ☐ 知道深层网络里前面参数也会影响最终 loss，所以也要更新。

6. 第 2 周：语言如何变成数字

神经网络只会处理数字，所以文本必须先变成 token id，再通过 embedding 变成向量。理解这一层以后，你看 nanoGPT 的数据准备、batch、block_size 会轻松很多。

6.1 字符级 tokenizer

```
text = "hello world"
chars = sorted(list(set(text)))
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}

def encode(s):
    return [stoi[c] for c in s]

def decode(ids):
    return ''.join(itos[i] for i in ids)

ids = encode("hello")
print(ids)
print(decode(ids))
```

6.2 Token id 不是语义，Embedding 才是可学习表示

token id 只是查表编号，比如 h -> 3。编号本身没有距离和语义。Embedding 表会把编号映射成向量，向量会参与训练，逐渐承载统计规律。

```
token_id -> embedding_table[token_id] -> vector
```

6.3 batch 和 block_size

语言模型训练时，x 是一段 token，y 是 x 右移一位。也就是说模型每个位置都在练习预测下一个 token。

```
import torch

text = "hello world, hello llm"
chars = sorted(list(set(text)))
stoi = {ch: i for i, ch in enumerate(chars)}
data = torch.tensor([stoi[c] for c in text], dtype=torch.long)

batch_size = 4
block_size = 5

ix = torch.randint(0, len(data) - block_size, (batch_size,))
x = torch.stack([data[i:i+block_size] for i in ix])
y = torch.stack([data[i+1:i+block_size+1] for i in ix])

print('x =')
print(x)
print('y =')
print(y)
```

6.4 第一个语言模型：Bigram

Bigram 模型只看上一个 token，虽然很弱，但它已经拥有语言模型的完整骨架：forward、loss、generate。

```
import torch
import torch.nn as nn
from torch.nn import functional as F

torch.manual_seed(1337)
text = "hello world, hello llm"
chars = sorted(list(set(text)))
vocab_size = len(chars)
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: ''.join([itos[i] for i in ids])
data = torch.tensor(encode(text), dtype=torch.long)

class BigramLanguageModel(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.token_embedding_table = nn.Embedding(vocab_size, vocab_size)

    def forward(self, idx, targets=None):
        logits = self.token_embedding_table(idx) # (B, T, C)
        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
            targets = targets.view(B*T)
            loss = F.cross_entropy(logits, targets)
        return logits, loss

    def generate(self, idx, max_new_tokens):
        for _ in range(max_new_tokens):
            logits, loss = self(idx)
            logits = logits[:, -1, :]
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, idx_next), dim=1)
        return idx

model = BigramLanguageModel(vocab_size)
idx = torch.zeros((1, 1), dtype=torch.long)
print(decode(model.generate(idx, max_new_tokens=20)[0].tolist()))
```

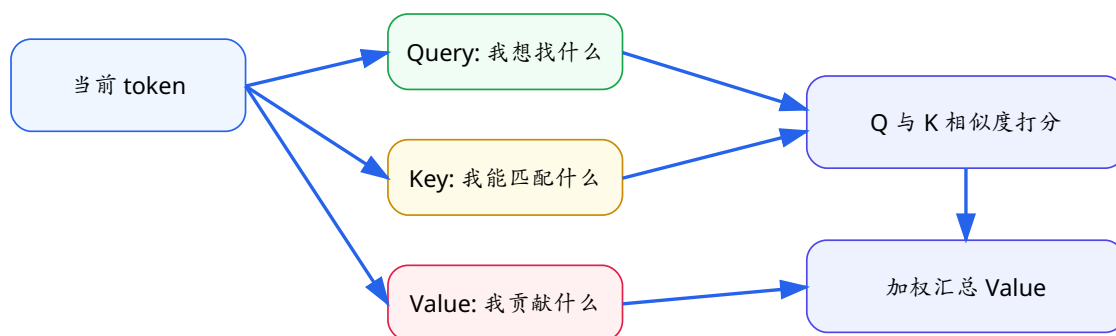
第 2 周检查点

- ☐ 能解释 token、token id、embedding 的区别。
- ☐ 能解释 dtype=torch.long 为什么常用于 token id。
- ☐ 能解释 x/y 右移一位的训练目标。
- ☐ 能解释 logits、softmax、cross entropy 的关系。
- ☐ 能跑通 BigramLanguageModel。

7. 第 3 周：Attention 与 Transformer

Attention 是 GPT 的核心。你可以先不要纠结公式，把它理解成：每个 token 都向上下文里的 token 发出查询，然后按相关程度取回信息。

Self-Attention 的 Q/K/V 直觉



一句话：Q/K 决定“看谁”，V 决定“拿走什么信息”。

图 4：Attention 的本质是让每个 token 从上下文里按权重取信息。

7.1 Attention 的计算顺序

1. 输入 x 分别乘以三个矩阵，得到 Q 、 K 、 V 。
2. Q 和 K 做相似度：谁和我相关？
3. 除以 $\sqrt{d}$ ，避免分数太大导致 softmax 太尖锐。
4. 使用 causal mask，防止当前位置看到未来。
5. softmax 得到权重。
6. 权重乘以 V ，汇总信息。

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

7.2 单个 Attention Head 代码

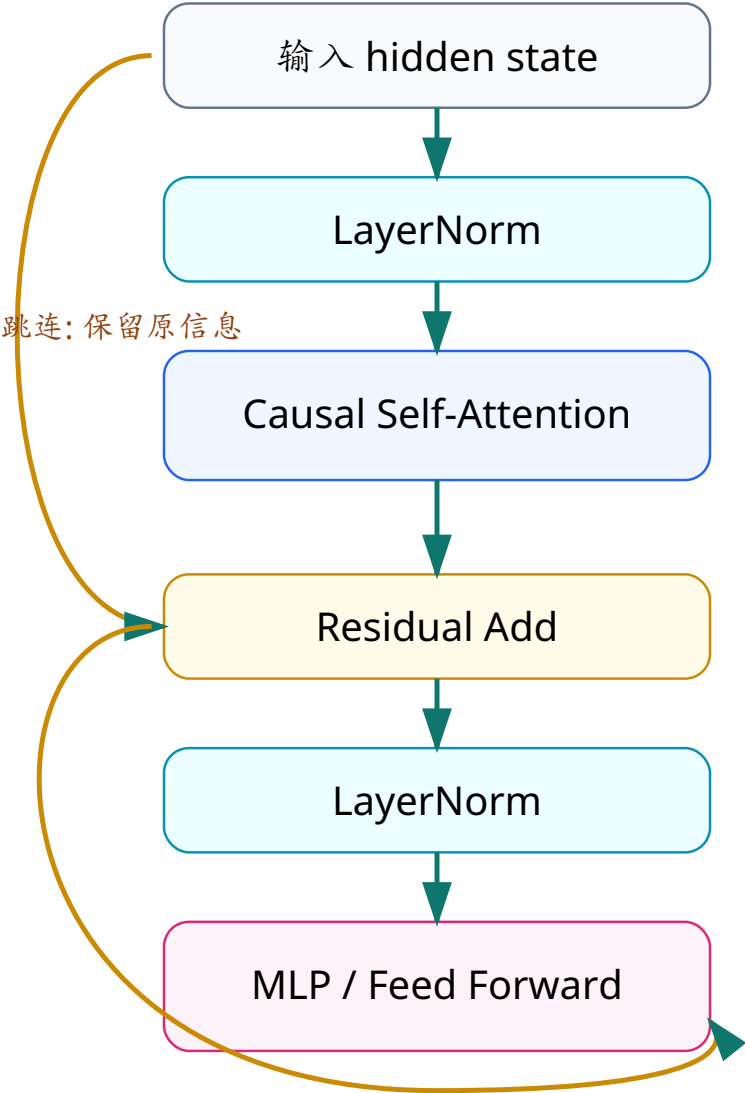
```
import torch
import torch.nn as nn
from torch.nn import functional as F

class Head(nn.Module):
    def __init__(self, n_embd, head_size, block_size, dropout=0.1):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x)          # (B, T, head_size)
        q = self.query(x)        # (B, T, head_size)
        wei = q @ k.transpose(-2, -1) * (k.shape[-1] ** -0.5) # (B, T, T)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
        wei = F.softmax(wei, dim=-1)
        wei = self.dropout(wei)
        v = self.value(x)
        out = wei @ v            # (B, T, head_size)
        return out
```

7.3 Transformer Block

GPT 的一个 Transformer Block



GPT = Embedding + N 个这样的 Block + 预测头。

图 5：Attention 负责混合上下文，MLP 负责对每个位置做非线性变换，Residual 让深层模型更容易训练。

模块	作用	一句话理解
Causal Self-Attention	让每个位置从前文取信息	解决看上下文的问题
Multi-Head	多个注意力视角并行	不同 head 可以关注不同关系
MLP / FFN	对每个位置做非线性变换	把取回的信息进一步加工
Residual	保留原始信息并改善梯度流动	不要每层都把旧信息洗掉
LayerNorm	稳定每层输入分布	让训练更稳

模块	作用	一句话理解
Position Embedding	提供顺序信息	否则模型不知道词的位置

第 3 周检查点

- ☐ 能解释 Q/K/V 分别是什么意思。
- ☐ 能解释 causal mask 为什么必要。
- ☐ 能写出单个 attention head。
- ☐ 能解释 multi-head 为什么是多个视角。
- ☐ 能画出 Transformer Block。

8. 第 4 周：手写 TinyGPT

第四周把前面所有零件组装起来。TinyGPT 的结构很简单：token embedding + position embedding + 多个 Transformer Block + lm_head。训练时算 loss，推理时 generate。

文件	建议内容
input.txt	训练文本。可以先放一篇文章或 Tiny Shakespeare。
train_tiny_gpt.py	完整训练脚本，包含模型、训练循环、生成。
notes/day23-training.md	记录训练 loss、val loss、生成样例。
experiments.md	记录 block_size、n_embd、n_head、n_layer 的实验结果。

8.1 TinyGPT 的张量形状

变量	形状	含义
idx	$B \times T$	一批 token id
tok_emb	$B \times T \times C$	token embedding
pos_emb	$T \times C$	position embedding
x	$B \times T \times C$	每个位置的 hidden state
logits	$B \times T \times \text{vocab_size}$	每个位置预测下一个 token 的分数

8.2 你应该做的 3 个实验

1. 固定其他参数，只改 block_size：观察上下文长度变大后速度和 loss 的变化。
2. 固定其他参数，只改 n_embd：观察模型宽度变大后的效果和速度。
3. 固定 n_embd，只改 n_layer/n_head：理解层数和 head 数对模型容量的影响。

不要追求生成内容很聪明

小数据、小模型、小训练步数下，生成结果很可能胡言乱语。你的目标是看懂链路和日志，而不是训练出可用聊天机器人。

9. 30 天每日计划

下面是每天的具体任务。建议每天结束后写 10 分钟笔记：今天的概念、跑了什么代码、遇到什么问题、明天要验证什么。

Day 1: LLM 的本质：预测下一个 token

- 原理** 建立总地图：LLM 不是数据库，而是一个把上下文映射成下一个 token 概率的函数。
- 实践** 画出 $\text{text} \rightarrow \text{token} \rightarrow \text{embedding} \rightarrow \text{transformer} \rightarrow \text{logits} \rightarrow \text{probability}$ 。
- 验收** 能用一句话解释 GPT 为什么能一个字一个字生成回答。

Day 2: 函数、参数、loss

- 原理** 理解模型为什么是复杂函数，参数是函数中可学习的旋钮，loss 是错得有多离谱。
- 实践** 手写 $y = w * x + b$ ，并用误差观察 w/b 改变后的影响。
- 验收** 能说清楚变量、参数、中间值、loss 的区别。

Day 3: 计算图与链式法则

- 原理** 把公式拆成节点，理解每个节点的输出如何影响最终 loss。
- 实践** 手算 $L = (a * b + c) * f$ 的 forward 和 backward。
- 验收** 能解释为什么 a 、 b 、 c 、 f 都会有 grad。

Day 4: 反向传播：为什么要往前传

- 原理** 理解不是只改最后一层，因为前面参数决定了后面拿到的特征。
- 实践** 运行简化 Value 类，打印每个变量的 grad。
- 验收** 能解释上上层参数为什么也要更新。

Day 5: 梯度正负与学习率

- 原理** 理解 grad 的正负不是好坏，而是 loss 对参数变化方向的提示。
- 实践** 用参数 $p = p - \text{lr} * \text{grad}$ 做 10 次更新。
- 验收** 能解释 $\text{grad} > 0$ 时参数为什么通常要减小。

Day 6: PyTorch 张量和自动求导

- 原理** 知道 tensor、dtype、requires_grad、backward 的基本含义。
- 实践** 写一个 torch 版线性回归小例子。
- 验收** 能解释 `torch.tensor(numbers, dtype=torch.long)` 为什么常用于 token id。

Day 7: 第 1 周复盘

- 原理** 把训练闭环串起来：forward、loss、backward、optimizer。
- 实践** 写一页笔记：模型如何从错误中变好。
- 验收** 不看资料讲出训练闭环。

Day 8: Token：文字如何进入模型

- 原理** 理解字符级 token、词级 token、BPE token 的区别。
- 实践** 实现字符级 encode/decode。
- 验收** 能解释 token id 只是编号，不是语义向量。

Day 9: Embedding：编号变向量

- 原理** 理解 embedding 是查表得到的可学习向量。
- 实践** 用 `nn.Embedding` 把 token id 转成向量。
- 验收** 能解释 embedding 为什么会在训练中变化。

Day 10: 训练样本：block_size 和 batch_size

- 原理** 理解语言模型训练样本如何从长文本切成 x/y。
- 实践** 实现 `get_batch`，打印 x 和 y。
- 验收** 能解释 y 为什么是 x 右移一位。

Day 11: Logits、Softmax、概率

- 原理** 理解模型最后输出的是每个 token 的分数，再转概率。
- 实践** 手写 softmax，观察大分数如何得到高概率。
- 验收** 能解释 logits 不是最终文字。

Day 12: Cross Entropy：预测错了如何惩罚

- 原理** 理解正确 token 概率越低，loss 越大。
- 实践** 用 `F.cross_entropy` 对几个 toy logits 计算 loss。
- 验收** 能解释为什么训练目标是降低正确下一个 token 的 loss。

Day 13: Bigram：第一个语言模型

- 原理** 先不碰 Transformer，用上一个 token 预测下一个 token。
- 实践** 实现 `BigramLanguageModel` 并生成文本。
- 验收** 知道 bigram 很弱，但它已经具备训练/生成闭环。

Day 14: Attention 直觉

- 原理** 理解每个 token 都从前文按权重取信息。
- 实践** 用中文写出 Q/K/V 三句话解释。
- 验收** 能解释 attention 是“看谁”和“拿什么”。

Day 15: Self-Attention 公式

- 原理** 理解 QK^T 、缩放、softmax、乘 V。
- 实践** 打印 attention weight 的形状和数值。
- 验收** 能画出 Q/K/V 流程。

Day 16: Causal Mask

- 原理** 理解 GPT 训练时不能偷看未来 token。
- 实践** 实现 torch.tril mask。
- 验收** 能解释为什么 GPT 是自回归模型。

Day 17: Multi-Head Attention

- 原理** 理解多个 head 像多个观察角度。
- 实践** 把多个 Head concat 后投影回 n_embd。
- 验收** 能解释 n_head 和 head_size 的关系。

Day 18: Position Embedding

- 原理** 理解如果没有位置信息，模型不知道 token 顺序。
- 实践** 实现 token embedding + position embedding。
- 验收** 能解释为什么“我爱你”和“你爱我”不一样。

Day 19: Transformer Block

- 原理** 理解 Attention、LayerNorm、Residual、MLP 的组合。
- 实践** 实现 Block 类。
- 验收** 能画出一个 GPT block。

Day 20: Residual 和 LayerNorm

- 原理** 理解深层模型为什么需要跳连和归一化。
- 实践** 对比 $x = \text{sa}(x)$ 和 $x = x + \text{sa}(\ln(x))$ 的结构。
- 验收** 知道它们不是核心语义模块，但对训练稳定很重要。

Day 21: TinyGPT 整体结构

- 原理** 把 embedding、blocks、ln_f、lm_head 串起来。
- 实践** 实现 TinyGPT.forward。
- 验收** 能解释 logits 的形状 B,T,vocab_size。

Day 22: Generate: 自回归生成

- 原理** 理解每次只预测下一个 token，再拼回上下文。
- 实践** 实现 generate 函数。
- 验收** 能解释 temperature/top-k 可以以后再学。

Day 23: 训练 TinyGPT

- 原理** 跑通完整训练循环。
- 实践** 准备 input.txt，运行 train_tiny_gpt.py。
- 验收** 能看懂 train loss 和 val loss。

Day 24: 实验 1: 上下文长度

- 原理** 理解 block_size 越大, 模型能看更多前文, 但 attention 更贵。
- 实践** 分别用 block_size=32/64/128 跑小实验。
- 验收** 记录 loss、速度和生成质量。

Day 25: 实验 2: 模型宽度和层数

- 原理** 理解 n_embd、n_layer、n_head 如何影响容量和速度。
- 实践** 改 n_embd/n_layer/n_head 并记录。
- 验收** 能解释模型变大不一定马上更好。

Day 26: 整理 TinyGPT 项目

- 原理** 把代码拆成 model.py/train.py/generate.py。
- 实践** 整理本地项目目录。
- 验收** 能从命令行训练和生成。

Day 27: 读 minGPT

- 原理** 用教育版 GPT 实现校准自己的代码理解。
- 实践** 重点读 CausalSelfAttention、Block、GPT.forward。
- 验收** 能把 minGPT 的 model.py 结构翻译成中文。

Day 28: 读 nanoGPT

- 原理** 理解更工程化的训练脚本、配置、日志。
- 实践** 重点读 model.py、train.py、config。
- 验收** 知道 nanoGPT 更偏训练工程, 不是第一份入门代码。

Day 29: 训练 vs 推理

- 原理** 理解训练要算 loss 和 backward, 推理只做 forward 和采样。
- 实践** 写一张训练/推理对比表。
- 验收** 能解释为什么推理不用反向传播。

Day 30: 终局复盘: 讲清楚 LLM

- 原理** 用一张图讲完整 LLM 原理。
- 实践** 完成最终口头讲解稿和项目 README。
- 验收** 达到: 能讲、能写、能跑、能改一个小 GPT。

10. 最终项目验收标准

第 30 天结束时，你应该交付一个本地小项目和一份 README。项目不需要工程化到线上部署，但必须能让你自己或别人复现。

项目结构

- input.txt
- train_tiny_gpt.py
- README.md
- experiments.md
- notes/30-day-summary.md

README 必须解释

- LLM 为什么是 next-token prediction
- 训练和推理区别
- TinyGPT 模型结构
- 关键超参数含义

代码必须能跑

- 安装依赖
- 启动训练
- 打印 loss
- 生成文本
- 修改超参数

口头讲解必须覆盖

- token/embedding
- attention
- transformer block
- loss/backward
- generate

最终自测问题

- ☐ 为什么模型输出的是 logits，而不是直接输出文字？
- ☐ 为什么 y 是 x 右移一位？
- ☐ 为什么 GPT 需要 causal mask？
- ☐ 为什么模型每次生成一个 token 后，还要把它拼回上下文？
- ☐ 训练时 backward 在做什么？推理时为什么不需要 backward？
- ☐ block_size 变大为什么会让 attention 更贵？
- ☐ n_layer、n_head、n_embd 分别改变了什么？

11. 附录 A：完整 TinyGPT 代码

把下面代码保存为 `train_tiny_gpt.py`，并在同目录准备 `input.txt`。第一次可以把任意一篇长文本复制进去；文本越长，训练越有意义。

```

# train_tiny_gpt.py
import torch
import torch.nn as nn
from torch.nn import functional as F

# ----- hyperparameters -----
batch_size = 32
block_size = 64
max_iters = 2000
eval_interval = 200
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 100
n_embd = 128
n_head = 4
n_layer = 4
dropout = 0.1
# -----

torch.manual_seed(1337)

with open('input.txt', 'r', encoding='utf-8') as f:
    text = f.read()

chars = sorted(list(set(text)))
vocab_size = len(chars)
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}
encode = lambda s: [stoi[c] for c in s]
decode = lambda ids: ''.join([itos[i] for i in ids])

data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9 * len(data))
train_data = data[:n]
val_data = data[n:]

def get_batch(split):
    data_src = train_data if split == 'train' else val_data
    ix = torch.randint(len(data_src) - block_size, (batch_size,))
    x = torch.stack([data_src[i:i+block_size] for i in ix])
    y = torch.stack([data_src[i+1:i+block_size+1] for i in ix])
    return x.to(device), y.to(device)

@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean().item()
    model.train()
    return out

class Head(nn.Module):
    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)

```

```

        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x)
        q = self.query(x)
        wei = q @ k.transpose(-2, -1) * (k.shape[-1] ** -0.5)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf'))
        wei = F.softmax(wei, dim=-1)
        wei = self.dropout(wei)
        v = self.value(x)
        return wei @ v

class MultiHeadAttention(nn.Module):
    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(n_embd, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        return self.dropout(self.proj(out))

class FeedForward(nn.Module):
    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd),
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd),
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

class Block(nn.Module):
    def __init__(self, n_embd, n_head):
        super().__init__()
        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedForward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

class TinyGPT(nn.Module):
    def __init__(self):
        super().__init__()
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd)
        self.lm_head = nn.Linear(n_embd, vocab_size)

    def forward(self, idx, targets=None):
        B, T = idx.shape

```

```

        tok_emb = self.token_embedding_table(idx)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device))
        x = tok_emb + pos_emb
        x = self.blocks(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)
        loss = None
        if targets is not None:
            B, T, C = logits.shape
            loss = F.cross_entropy(logits.view(B*T, C), targets.view(B*T))
        return logits, loss

    def generate(self, idx, max_new_tokens):
        for _ in range(max_new_tokens):
            idx_cond = idx[:, -block_size:]
            logits, loss = self(idx_cond)
            logits = logits[:, -1, :]
            probs = F.softmax(logits, dim=-1)
            idx_next = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, idx_next), dim=1)
        return idx

model = TinyGPT().to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

for iter in range(max_iters):
    if iter % eval_interval == 0:
        losses = estimate_loss()
        print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")
    xb, yb = get_batch('train')
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

context = torch.zeros((1, 1), dtype=torch.long, device=device)
print(decode(model.generate(context, max_new_tokens=400)[0].tolist()))

```

运行方式

```

cd ~/src/me/learnAIagent/llm-30days
source .venv/bin/activate
python train_tiny_gpt.py

```


实验记录模板

```
# experiments.md

## 实验 1: block_size
- 日期:
- 参数: batch_size=32, block_size=64, n_embd=128, n_head=4, n_layer=4
- step 0 loss:
- step 200 loss:
- step 1000 loss:
- 生成样例:
- 我的结论:

## 实验 2: n_embd
- 改动: 128 -> 256
- 速度变化:
- loss 变化:
- 我的结论:
```

12. 附录 B：资源与延伸阅读

下面资源建议按顺序使用。30 天内先看能帮助你完成主线的部分，不要陷入无止境收藏资料。

资源	用途	链接
Karpathy - Neural Networks: Zero to Hero	从反向传播到 GPT 的实践课程	https://karpathy.ai/zero-to-hero.html
karpathy/minGPT	教育版 GPT 实现，小而清晰	https://github.com/karpathy/minGPT
karpathy/nanoGPT	更工程化的 GPT 训练仓库	https://github.com/karpathy/nanoGPT
Attention Is All You Need	Transformer 原始论文	https://arxiv.org/abs/1706.03762
The Illustrated Transformer	图解 Transformer 和 Attention	https://jalammar.github.io/illustrated-transformer/
PyTorch Get Started	安装 PyTorch 的官方入口	https://pytorch.org/get-started/locally/
Dive into Deep Learning	带代码和数学的深度学习在线书	https://d2l.ai/
Hugging Face LLM Course	理解 Transformers、Tokenizers、Datasets 的实践课程	https://huggingface.co/learn/llm-course/

下一阶段：学 Agent 前需要补什么

这 30 天结束后，再进入 Agent 会更顺。下一阶段建议学习：RAG、Tool Calling、Function Calling、Memory、Planning、LangGraph、评测与监控。因为你已经知道 LLM 本身只是 next-token predictor，Agent 才是把模型、工具、状态、外部知识和流程控制组织起来的系统。

文档生成日期：2026-06-14。建议配合本地代码实验使用。